Your grade: 85.71%

Your latest: **85.71%** • Your highest: **85.71%** • To pass you need at least 71%. We keep your highest score.

Next item →

Skill progress from this assignment Beta

Advanced RAG System Implementation +15 XP
163/480 XP

---

1. In the context of Retrieval-Augmented Generation (RAG), which step does LlamaIndex combine with the prompt augmentation process? 0 / 1 point

- LLM response generation
- Vector storage
- Document chunking
- User prompt acceptance

Try again

Review the video 'Intro to LlamaIndex: From Vector Stores to Query Engines'.

2. What is the purpose of LlamaIndex's `LangChainNodeParser`? 1 / 1 point

- To embed user prompts
- To load the directories of files
- To store vectors in memory
- To wrap around LangChain's text splitters

Nice work

LlamaIndex's `LangChainNodeParser` wraps around LangChain's text splitters, allowing their use within LlamaIndex.

3. What is the main purpose of embedding the user's original prompt in LlamaIndex? 1 / 1 point

- To load the prompt into a directory
- To store the prompt in memory
- To retrieve relevant chunks from the vector store
- To split the prompt into smaller chunks

Nice work

Embedding the user's original prompt is typically done to retrieve relevant chunks from the vector store, ensuring the embedding model matches the one used for chunk embeddings.

4. Which LangChain splitter is most similar to LlamaIndex's `SentenceSplitter`? 1 / 1 point

- `TokenTextSplitter`
- `CharacterTextSplitter`
- `RecursiveCharacterTextSplitter`
- `SemanticChunker`

Nice work

LlamaIndex's `SentenceSplitter` recursively splits text based on predefined characters. By default, `SentenceSplitter` splits text by paragraphs, where paragraphs are identified by multiple newline characters in succession. If some of the resulting chunks are still too large, those chunks get split using secondary splitters, such as periods and commas. In spirit this is most similar to LangChain's

5. What is the purpose of LlamaIndex's "prompt synthesizer"? 1 / 1 point

- To perform prompt augmentation internally and pass the augmented prompt to the LLM
- To manually pass the augmented prompt to the LLM
- To retrieve nodes from the vector store
- To combine prompt augmentation with query embedding

Nice work

The "prompt synthesizer" in LlamaIndex takes the user's original prompt and retrieved nodes as inputs, performs prompt augmentation internally, and passes the augmented prompt to the LLM.

6. What is a key feature of LlamaIndex's `VectorStoreIndex`? 1 / 1 point

- It does not support vector embeddings
- It requires manual setup of metadata
- It only works with in-memory storage
- It integrates external vector databases into a native class

Nice work

LlamaIndex's `VectorStoreIndex` wraps external vector databases into a native class, making downstream code agnostic to the vector store backend.

7. Anna wants to develop a RAG system with minimal reliance on external libraries. She prefers a framework with robust internal capabilities for tasks like document loading and in-memory vector storage, while still allowing integration with external libraries and services when needed. 1 / 1 point

Which framework, LangChain or LlamaIndex, best suits Anna's needs, and why?

- LangChain, because LlamaIndex is incapable of building a RAG system
- LangChain, because it provides superior native solutions to document loading and vector storage than LlamaIndex
- LlamaIndex, because LangChain is incapable of building a RAG system
- LlamaIndex, because it provides superior native solutions to document loading and vector storage than LangChain

Nice work

LlamaIndex has superior native tools than LangChain for most tasks but still allows numerous integrations with external libraries. LangChain, on the other hand, tends to rely more on external integrations. This does not imply that one framework is better overall; rather, they are different approaches taken by different projects. However, if one's objective was to minimize reliance on external libraries, LlamaIndex would be the correct choice because more functionality is built into LlamaIndex's core library.